1. Aim

To implement a **Recurrent Neural Network (RNN)** and **Long Short-Term Memory (LSTM)** model for handling sequential data such as **time series prediction or text classification**, and to compare their performance.

---

2. Theory

Recurrent Neural Network (RNN)

RNNs are designed for sequential data where current output depends on previous computations. They maintain a **hidden state** that captures past information.

Key Idea:

ht=f(Whht−1+Wxxt)h_t = f(W_h h_{t-1} + W_x x_t)ht=f(Whht−1+Wxxt)

- hth_tht: hidden state
- xtx_txt: input at time step ttt
- Wh,WxW_h, W_xWh,Wx: weight matrices

Limitation:

- Suffers from **vanishing and exploding gradient problem**, making it hard to learn long-term dependencies.

Long Short-Term Memory (LSTM)

LSTM is a special type of RNN designed to overcome RNN limitations.

Components:

- **Forget Gate**: decides what to discard
- **Input Gate**: decides what to store
- **Output Gate**: decides output

Advantages:

- Captures long-term dependencies
- Reduces vanishing gradient problem

3. Procedure

1. Import required libraries
2. Load and preprocess dataset
3. Convert data into sequences
4. Split into training and testing sets
5. Build RNN/LSTM model
6. Compile model
7. Train the model
8. Evaluate performance
9. Make predictions

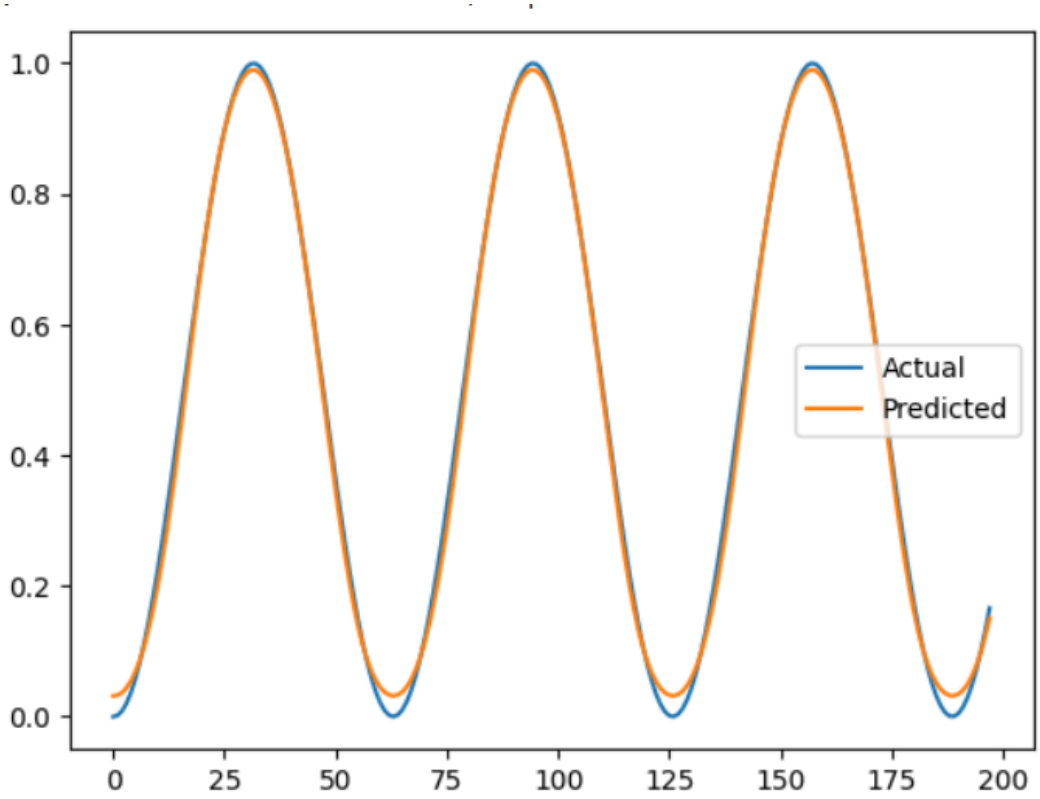
A. Time Series Prediction using LSTM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM
from sklearn.preprocessing import MinMaxScaler

# Load dataset (example: sine wave)
data = np.sin(np.arange(0, 100, 0.1))

# Normalize data
scaler = MinMaxScaler()
data = scaler.fit_transform(data.reshape(-1, 1))

# Create sequences
def create_dataset(dataset, time_step=10):
    X, y = [], []
    for i in range(len(dataset) - time_step - 1):
        X.append(dataset[i:(i + time_step), 0])
        y.append(dataset[i + time_step, 0])
    return np.array(X), np.array(y)

time_step = 10
X, y = create_dataset(data, time_step)

# Reshape input for LSTM [samples, time steps, features]
X = X.reshape(X.shape[0], X.shape[1], 1)
```

```
# Split data
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Build LSTM model
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_step, 1)))
model.add(LSTM(50))
model.add(Dense(1))

# Compile model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train model
model.fit(X_train, y_train, epochs=10, batch_size=32)

# Predictions
predictions = model.predict(X_test)

# Plot results
plt.plot(y_test, label="Actual")
plt.plot(predictions, label="Predicted")
plt.legend()
plt.show()
```

Output

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential mode
  super().__init__(**kwargs)
25/25 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1198
Epoch 2/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0387
Epoch 3/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0317
Epoch 4/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0273
Epoch 5/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0218
Epoch 6/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0134
Epoch 7/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0034
Epoch 8/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0011
Epoch 9/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 5.3589e-04
Epoch 10/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 3.2759e-04
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step
```



B. Simple RNN for Text Classification

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)

# Pad sequences
X_train = pad_sequences(X_train, maxlen=100)
X_test = pad_sequences(X_test, maxlen=100)

# Build RNN model
model = Sequential()
```

```
model.add(Embedding(10000, 32, input_length=100))
model.add(SimpleRNN(32))
model.add(Dense(1, activation='sigmoid'))

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=5, batch_size=64)

# Evaluate
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

Output:

```
Epoch 1/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 13s 26ms/step - accuracy: 0.6118 - loss: 0.6423
Epoch 2/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 10s 26ms/step - accuracy: 0.8434 - loss: 0.3659
Epoch 3/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 21s 27ms/step - accuracy: 0.9268 - loss: 0.1961
Epoch 4/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 11s 27ms/step - accuracy: 0.9737 - loss: 0.0858
Epoch 5/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 10s 25ms/step - accuracy: 0.9905 - loss: 0.0365
782/782 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - accuracy: 0.8114 - loss: 0.6159
Accuracy: 0.8113600015640259
```

5. Results

- **LSTM performs better for time series due to long-term memory capability**
- **RNN works for short sequences but struggles with long dependencies**

6. Conclusion

- **RNNs are useful for sequential data but have limitations with long-term dependencies**
- **LSTMs overcome these issues using gating mechanisms**
- **For most real-world applications (text, speech, time series), LSTM is preferred over basic RNN**

7. Applications

- **Time series forecasting (stock prices, weather)**
- **Natural Language Processing (sentiment analysis, translation)**
- **Speech recognition**
- **Sequence prediction**